LAPORAN PROYEK
HIFICHAIN: SISTEM AUTENTIKASI ASET AUDIO TERDESENTRALISASI

Disusun sebagai pemenuhan Tugas Besar
IF4035 Blockchain

Oleh
NAUFAL BALDEMAR ARDANNI
NIM: 13521154

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO & INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Februari 2026

DAFTAR ISI

BAB I PENDAHULUAN.....1

I.1 Latar Belakang.....1

I.2 Rumusan Masalah.....1

I.3 Tujuan.....2

I.4 Deskripsi Use Case.....2

BAB II KAJIAN PUSTAKA.....4

II.1 Blockchain.....4

II.2 Ethereum dan EVM.....4

II.3 Kontrak Pintar dan Solidity.....4

II.4 Hardhat.....5

II.5 Oracle.....5

II.6 Pengembangan Antarmuka (React dan Ethers.js).....5

II.7 Standar Keamanan OWASP Smart Contract.....6

BAB III ANALISIS DAN PERANCANGAN SISTEM.....7

III.1 Arsitektur Sistem.....7

III.2 Desain Kontrak Pintar.....7

III.4 Analisis dan Integrasi Oracle.....9

III.4.2 Mekanisme dan Diagram Sekuens.....10

III.4.3 Skenario Buruk dan Mitigasinya (Fail-Safe).....11

III.5 Pola Desain Kontrak Pintar.....12

III.6 Optimasi Kontrak.....12

BAB IV IMPLEMENTASI SISTEM.....14

IV.1 Lingkungan Implementasi.....14

IV.2 Implementasi Kontrak Pintar.....14

IV.3 Implementasi Oracle Service.....16

IV.4 Implementasi Antarmuka Pengguna.....16

IV.4.1 Halaman Pendaftaran (Minting).....16

IV.4.2 Halaman Verifikasi.....17

BAB V PENGUJIAN DAN ANALISIS.....19

V.1 Skenario Pengujian (Black Box Testing).....19

V.2 Analisis Keamanan (OWASP Smart Contract Top 10 2025).....20

V.2.1 Access Control Vulnerabilities.....20

V.2.2 Integer Overflow.....21

V.2.3 Oracle Manipulation.....21

BAB VI PENUTUP.....22

VI.1 Daftar Anggota dan Pembagian Tugas.....22

VI.2 Saran Pengembangan.....22

DAFTAR GAMBAR

Gambar I.1. Diagram *Use Case*……...……3

Gambar III.1. Diagram Kelas Kontrak Pintar……...……...8

Gambar III.2. Diagram Sekuens……...……..11

Gambar IV.1. Struktur Kode Kontrak Pintar HiFiChain.sol……...……...…15

Gambar IV.2. Log Oracle Mengakuisisi Harga ETH Eksternal…….....16

Gambar IV.3. Notifikasi Sukses Pendaftaran Aset…….....17

Gambar IV.4. Tampilan Sertifikat Digital Valid……..……...…18

Gambar V.1 Pesan Galat Saat Akun Tidak Berwenang…….21

DAFTAR TABEL

Tabel III.1 Risiko Kegagalan…….…….11

Tabel V.1 Hasil Pengujian……...…19

Tabel VI.1 Daftar Anggota dan Pembagian Tugas…….…22

BAB I

PENDAHULUAN

I.1 Latar Belakang

Industri perangkat audio *high-fidelity* (HiFi) merupakan pasar yang bernilai tinggi. Perangkat seperti penyuara jemala (*headphone*), *amplifier*, dan DAC sering kali diperdagangkan dengan harga yang mencapai ribuan hingga puluhan ribu dolar Amerika Serikat (USD). Tingginya nilai aset ini memunculkan masalah yang signifikan, yakni peredaran barang palsu (*counterfeit*) dan penipuan di pasar sekunder (*second-hand market*).

Sistem verifikasi konvensional yang mengandalkan sertifikat kertas atau basis data terpusat memiliki kelemahan yang fundamental. Sertifikat fisik mudah untuk dipalsukan, sementara basis data terpusat rentan terhadap manipulasi data oleh pihak internal ataupun serangan siber eksternal (*single point of failure*). Konsumen sering kali tidak memiliki cara independen untuk memverifikasi apakah perangkat yang mereka beli adalah aset autentik yang diproduksi oleh pabrik yang resmi.

Teknologi *blockchain* menawarkan solusi melalui sifatnya yang tidak dapat diubah (*immutable*) dan terdesentralisasi. Dengan mencatat identitas aset (nomor seri dan model) ke dalam *ledger* yang terdistribusi, riwayat kepemilikan dan keaslian barang dapat diverifikasi secara transparan tanpa bergantung pada kepercayaan terhadap suatu entitas terpusat. Sistem yang dikembangkan bernama "HiFiChain", sebuah aplikasi terdesentralisasi (*dApp*) yang memanfaatkan kontrak pintar Ethereum dan integrasi *oracle* untuk menciptakan sistem autentikasi aset yang aman, transparan, dan tepercaya.

I.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah yang diselesaikan dalam tugas besar ini adalah sebagai berikut:

1. Bagaimana membangun sistem pencatatan aset yang *tamper-proof* sehingga sertifikat keaslian tidak dapat dipalsukan atau dihapus?

2. Bagaimana mekanisme verifikasi yang memungkinkan pengguna umum memeriksa keaslian aset tanpa perlu log masuk atau izin khusus?
3. Bagaimana mengintegrasikan data dunia nyata (harga pasar ETH/USD) ke dalam lingkungan *blockchain* yang tertutup untuk memberikan konteks nilai pada aset?

I.3 Tujuan

Tujuan dari pengembangan HiFiChain adalah

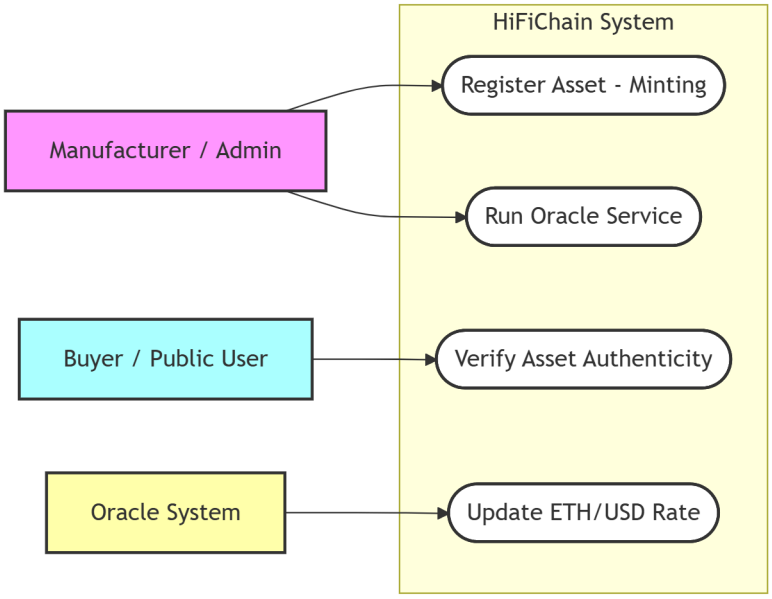
1. mengimplementasikan smart contract pada private blockchain untuk pencatatan (minting) aset audio;
2. membangun antarmuka (frontend) berbasis web yang ramah pengguna untuk interaksi dengan blockchain;
3. mengimplementasikan mekanisme keamanan kontrol akses untuk mencegah pihak tidak berwenang memanipulasi data aset; serta
4. mengembangkan layanan oracle sederhana untuk mengambil data harga eksternal dan menyimpannya secara on-chain.

I.4 Deskripsi Use Case

Sistem HiFiChain melibatkan tiga aktor utama dengan peran sebagai berikut:

1. ***Manufacturer* (Admin)**: Memiliki otoritas penuh untuk mendaftarkan aset baru (*minting*) dan menjalankan layanan *oracle*.
2. ***Buyer* (*Public User*)**: Pengguna umum yang dapat memverifikasi keaslian aset dengan menggunakan nomor seri.
3. ***Oracle System***: Aktor sistem (otomatis) yang bertugas memperbarui nilai tukar ETH/USD secara berkala.

Gambar I.1 mengilustrasikan *use case* yang dapat dilakukan oleh ketiga aktor tersebut.



Gambar I.1 Diagram *Use Case*

BAB II

KAJIAN PUSTAKA

II.1 Blockchain

Blockchain adalah teknologi *ledger* terdistribusi yang mencatat transaksi secara permanen, aman, dan transparan. Berbeda dengan basis data tradisional yang dikelola oleh satu entitas pusat, *blockchain* dikelola oleh jaringan *node* (komputer) yang bekerja sama untuk memvalidasi dan menyimpan data. Tiap blok data baru dihubungkan secara kriptografis dengan blok sebelumnya dengan menggunakan *hash* sehingga membentuk rantai (*chain*) yang membuat data historis hampir mustahil untuk diubah (*immutable*).

Dalam konteks manajemen aset, *blockchain* menawarkan solusi untuk masalah kepercayaan dan transparansi. Riwayat kepemilikan dan keaslian barang dapat diverifikasi oleh siapa saja tanpa perantara.

II.2 Ethereum dan EVM

Ethereum adalah platform *blockchain* sumber terbuka yang memungkinkan pengembang untuk membangun dan menyebarkan (*deploy*) aplikasi terdesentralisasi (*dApp*). Fitur utama Ethereum adalah kemampuannya menjalankan kontrak pintar.

Ethereum Virtual Machine (EVM) adalah lingkungan komputasi (*runtime*) yang berjalan pada tiap node Ethereum. EVM bertugas untuk mengeksekusi bytecode dari kontrak pintar dan memperbarui state jaringan setelah tiap transaksi.

II.3 Kontrak Pintar dan Solidity

Kontrak pintar adalah program komputer yang berjalan di atas *blockchain*. Program ini berisi logika bisnis yang dieksekusi secara otomatis ketika kondisi tertentu terpenuhi. Sifat kontrak pintar adalah deterministik (hasil eksekusi selalu sama di tiap node) dan *immutable* (kode tidak dapat diubah setelah disebarkan).

Solidity adalah bahasa pemrograman berorientasi objek tingkat tinggi yang dirancang secara khusus untuk menulis kontrak pintar pada platform EVM. Solidity memiliki sintaksis yang mirip dengan JavaScript dan C++ sehingga mendukung pewarisan (*inheritance*), pustaka (*libraries*), dan tipe data yang kompleks.

II.4 Hardhat

Hardhat adalah lingkungan pengembangan untuk Ethereum yang digunakan untuk mengompilasi, menyebarkan, menguji, dan men-*debug* aplikasi *blockchain*. Hardhat menyediakan Hardhat Network, sebuah jaringan Ethereum lokal yang dirancang untuk pengembangan. Jaringan ini menyimulasikan perilaku Ethereum Mainnet sehingga memungkinkan pengembang untuk melakukan transaksi yang instan, *debugging stack traces*, dan pencatatan log konsol langsung dari kode Solidity.

Dalam tugas besar ini, Hardhat digunakan sebagai infrastruktur *private blockchain* untuk menjalankan simulasi sistem tanpa memerlukan biaya transaksi gas yang nyata (ETH).

II.5 Oracle

Blockchain bersifat tertutup dan deterministik. Artinya, kontrak pintar tidak dapat mengakses data dari dunia luar (*off-chain*) secara langsung (seperti harga saham, cuaca, atau data API). *Oracle* adalah entitas atau layanan yang bertindak sebagai jembatan yang menghubungkan *blockchain* dengan sumber data eksternal.

Oracle bekerja dengan cara memantau permintaan data dari *blockchain*, mengakuisisi data tersebut dari sumber eksternal (misalnya API web), dan mengirimkan data tersebut kembali ke kontrak pintar melalui sebuah transaksi. Tanpa Oracle, kegunaan *blockchain* akan terbatas hanya pada data yang dihasilkan di dalam jaringannya sendiri.

II.6 Pengembangan Antarmuka (React dan Ethers.js)

Untuk memungkinkan pengguna berinteraksi dengan blockchain, diperlukan antarmuka pengguna (Frontend) yang terhubung dengan dompet digital.

- **React.js (Vite)**: Pustaka JavaScript populer untuk membangun antarmuka pengguna yang reaktif dan berbasis komponen. Vite digunakan sebagai *build tool* untuk performa pengembangan yang cepat.
- **Ethers.js**: Pustaka JavaScript yang menyediakan abstraksi lengkap untuk berinteraksi dengan *blockchain* Ethereum. Ethers.js memungkinkan aplikasi web untuk menghubungkan dompet (seperti MetaMask), membaca data dari kontrak pintar, dan menandatangani transaksi kriptografis.

II.7 Standar Keamanan OWASP Smart Contract

Open Web Application Security Project (OWASP) menyediakan panduan keamanan untuk pengembangan kontrak pintar. Beberapa kerentanan utama yang harus dimitigasi tercantum sebagai berikut:

1. ***Access Control Vulnerabilities***: Kegagalan dalam membatasi akses ke fungsi kritis yang memungkinkan penyerang mengambil alih kontrak.
2. ***Integer Overflow/Underflow***: Kesalahan perhitungan aritmatika yang melebihi batas penyimpanan variabel. (Secara otomatis ditangani oleh Solidity v0.8+).
3. ***Reentrancy Attacks***: Serangan yang memungkinkan kontrak eksternal untuk memanggil kembali fungsi sebelum eksekusi awal selesai.

BAB III

ANALISIS DAN PERANCANGAN SISTEM

III.1 Arsitektur Sistem

HiFiChain dibangun dengan menggunakan arsitektur aplikasi terdesentralisasi (*dApp*) yang terdiri atas tiga lapisan utama, yaitu (1) *presentation layer*, (2) *logic layer*, dan (3) *data layer*.

1. ***Frontend* (*Presentation Layer*)**
    a. Dibangun dengan menggunakan React dan Vite.
    b. Bertugas untuk menangani interaksi pengguna dan menghubungkan dompet digital (MetaMask) dengan menggunakan pustaka (*library*) Ethers.js.
2. ***Smart Contract* (*Logic Layer*)**
    a. Ditulis dalam bahasa Solidity (v0.8.28).
    b. Berjalan di atas Hardhat Network (EVM lokal).
    c. Berisi logika bisnis utama, yakni registrasi aset, verifikasi kepemilikan, dan manajemen data *oracle*.
3. ***Oracle Service***
    a. *Script* Node.js yang berjalan secara independen.
    b. Mengakuisisi data harga ETH/USD dari API CoinGecko dan mengirimkan transaksi pembaruan ke *smart contract*.

III.2 Desain Kontrak Pintar

III.2.1 Desain Diagram Kelas Kontrak Pintar

Kontrak HiFiChain dirancang sebagai kelas tunggal yang menyimpan state aset dan logika bisnis. Gambar III.1 mengilustrasikan diagram kelas dari kontrak pintar. Berikut adalah properti dan metode utama:

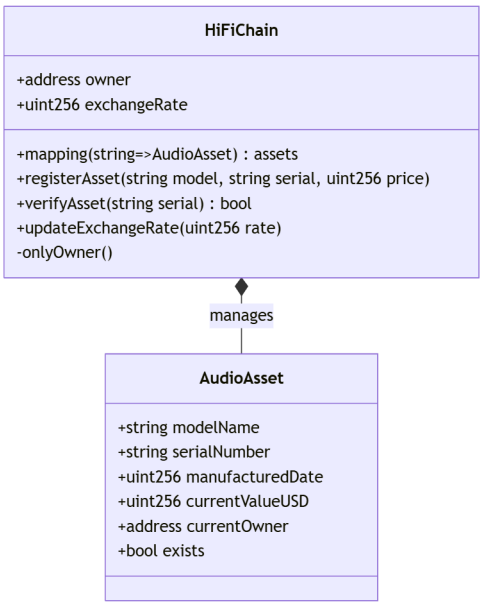
Perincian Properti:

- `owner: address`: Menyimpan alamat *deployer* untuk validasi akses.

- `exchangeRate: uint256`: Menyimpan harga ETH terkini (dari *oracle*).
- `assets: mapping(string => AudioAsset)`: *Hash map* untuk penyimpanan data aset (kompleksitas waktu akses O(1)).

Perincian *Method*:

- `registerAsset(...)`: Menulis data aset baru ke *blockchain* (Hanya *Owner*).
- `verifyAsset(...)`: Membaca data aset berdasarkan nomor serial (Publik).
- `updateExchangeRate(...)`: Memperbarui variabel exchangeRate (Hanya *Owner*).



Gambar III.1 Diagram Kelas Kontrak Pintar

III.2.2 Struktur Data (Struct)

Entitas utama dalam sistem adalah AudioAsset, yang merepresentasikan satu unit perangkat audio.

```
struct AudioAsset {
    string modelName; // Nama Model
```

```
    string serialNumber; // Primary Key (Unik)
    uint256 manufacturedDate; // Timestamp pencatatan
    uint256 currentValueUSD; // Nilai aset saat registrasi
    address currentOwner; // Alamat wallet pabrikan/pemilik
    bool exists; // Flag validasi keberadaan data
}
```

III.2.3 Mekanisme Keamanan (Kontrol Akses)

Untuk memenuhi standar keamanan OWASP Smart Contract, sistem menerapkan pola `onlyOwner`. Variabel `address public owner` menyimpan alamat penyebar kontrak. *Modifier* `onlyOwner` diaplikasikan pada fungsi krusial:

- `registerAsset()`: Mencegah pengguna umum mendaftarkan barang palsu.
- `updateExchangeRate()`: Mencegah manipulasi harga oleh pihak ketiga.

III.3 Alur Kerja Sistem

1. ***Deploy***: Admin menyebarkan kontrak ke jaringan Hardhat.
2. ***Oracle Update***: Layanan Oracle mengakuisisi harga ETH terbaru dan memperbarui *state* `exchangeRate` di kontrak.
3. **Registrasi**: Admin menginput data aset via *frontend* → MetaMask menandatangani transaksi → Data disimpan di *Blockchain*.
4. **Verifikasi**: Pembeli memasukkan nomor seri → *Frontend* memanggil fungsi `verifyAsset()` (*call*, bukan transaksi) → Kontrak mengembalikan data autentik jika ditemukan.

III.4 Analisis dan Integrasi Oracle

Sistem menggunakan pola *off-chain oracle* sederhana untuk menjembatani data dunia nyata ke dalam *blockchain*.

III.4.1 Struktur Data dan API

Data diakuisisi dari API CoinGecko dengan *endpoint* /simple/price. Struktur JSON yang diterima adalah sebagai berikut:

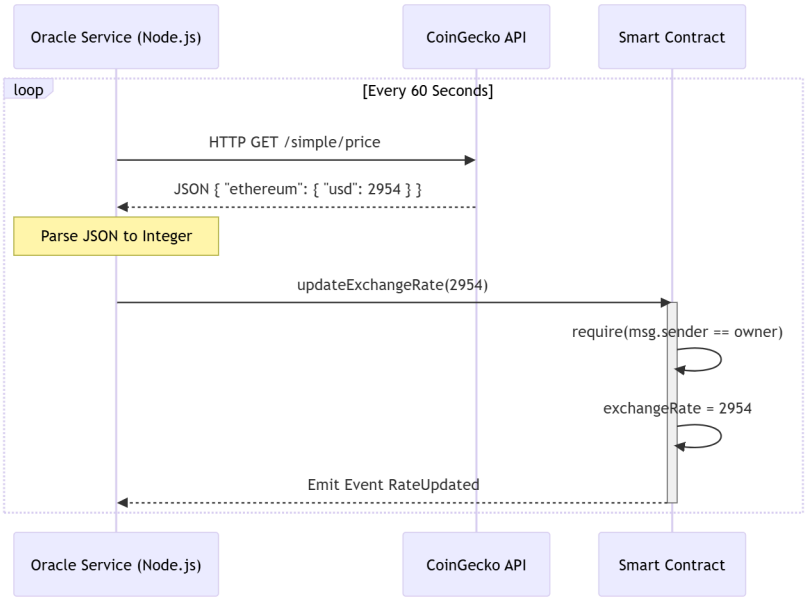
```
{
  "ethereum": {
    "usd": 2954
  }
}
```

Layanan *oracle* melakukan *parsing* pada objek ethereum.usd, mengubahnya menjadi integer (untuk kompatibilitas Solidity), dan mengirimkannya ke kontrak.

III.4.2 Mekanisme dan Diagram Sekuens

Proses pengambilan data berjalan secara otomatis menggunakan *script* Node.js. Proses tersebut terangkum pada Gambar III.2 dengan tahapan sebagai berikut:

1. *Oracle service* mengirim request HTTP GET ke API CoinGecko.
2. CoinGecko mengembalikan respons JSON harga ETH.
3. *Oracle service* membentuk transaksi Ethereum dengan memanggil fungsi `updateExchangeRate(price)`.
4. Kontrak pintar memvalidasi pengirim (`onlyOwner`).
5. Kontrak pintar memperbarui state `exchangeRate` dan memancarkan *event*.



Gambar III.2 Diagram Sekuens

III.4.3 Skenario Buruk dan Mitigasinya (*Fail-Safe*)

Dalam implementasi Oracle terpusat, terdapat beberapa risiko kegagalan yang tercantum dalam Tabel III.1.

Tabel III.1. Risiko Kegagalan

Skenario Buruk	Dampak	Solusi/Mitigasi
API *Down*/*Timeout*	Data harga tidak ter-*update* (*stale*).	***Error Handling* (*Try-Catch*)**: Skrip Oracle menggunakan blok *try-catch* agar layanan tidak *crash* total saat API gagal dan akan mencoba lagi pada interval berikutnya.
Manipulasi *Oracle*	Pihak ketiga mengirim harga palsu (misal: ETH = $0).	**Kontrol Akses (SC02:2025)**: Kontrak pintar menerapkan `onlyOwner`. Hanya dompet resmi manufaktur yang diizinkan memperbarui harga.
Input Data Nol	Harga 0 merusak valuasi aset.	**Validasi Input (SC04:2025)**: Kontrak memiliki `require(_rate > 0)` untuk menolak pemutakhiran jika harga bernilai nol.

III.5 Pola Desain Kontrak Pintar

Dalam pengembangan HiFiChain, digunakan beberapa *software design pattern* standar Solidity:

1. ***Access Control Pattern* (*Guard Check*):**
    - **Implementasi**: *Modifier* `onlyOwner`.
    - **Alasan**: Membatasi akses fungsi kritis (`register`, `updateRate`) hanya kepada entitas yang berwenang (manufaktur) untuk mencegah perubahan *state* yang *unauthorized*.
2. ***Checks-Effects-Interactions Pattern*:**
    - **Implementasi**: Fungsi `registerAsset` melakukan validasi (`require`) terlebih dahulu sebelum menulis ke memori (`assets[...] = ...`) dan memancarkan *event*.
    - **Alasan**: Mencegah perilaku tak terduga dan memitigasi potensi serangan *reentrancy* (meskipun risikonya rendah pada kontrak ini).
3. ***Oracle Pattern* (*Inbound Oracle*):**
    - **Implementasi**: Pemisahan logika pengambilan data (*off-chain script*) dan penyimpanan data (*on-chain variable*).
    - **Alasan**: Karena *blockchain* bersifat deterministik dan terisolasi, data eksternal harus "didorong" masuk (*push*) oleh entitas luar, bukan "ditarik" (*pull*) oleh kontrak.

III.6 Optimasi Kontrak

Beberapa teknik optimasi gas dan efisiensi diterapkan pada kode:

1. **Penggunaan *Mapping* vs. Larik**: Penyimpanan aset menggunakan `mapping(string => AudioAsset)` alih-alih larik. Pilihan ini memungkinkan akses data (verifikasi) dengan kompleksitas waktu O(1) (konstan), yang jauh lebih murah biaya gas-nya dibandingkan iterasi *looping* pada larik (O(n)).
2. ***Built-in Overflow Protection***: Penggunaan Solidity v0.8.28 meniadakan kebutuhan pustaka eksternal SafeMath sehingga mengurangi ukuran *bytecode* kontrak dan biaya *deployment* menjadi lebih murah.

3. ***Event Indexing***: Parameter `indexed` digunakan pada `Event AssetRegistered` sehingga memungkinkan pencarian log yang efisien oleh aplikasi *frontend* tanpa membebani penyimpanan kontrak utama.

BAB IV

IMPLEMENTASI SISTEM

Bab ini mendeskripsikan rencana pelaksanaan tugas akhir, yang meliputi jadwal pengerjaan serta risiko-risiko yang mungkin dihadapi dan rencana mitigasinya.

IV.1 Lingkungan Implementasi

Pengembangan dan implementasi sistem HiFiChain dilakukan pada lingkungan lokal dengan spesifikasi perangkat lunak sebagai berikut:

- *Runtime Environment*: Node.js v18.0.0
- *Blockchain Framework*: Hardhat (*Localhost Network*)
- Bahasa Pemrograman: Solidity v0.8.28 (*backend*) serta JavaScript (*frontend* dan *oracle*)
- Dompet: MetaMask *Browser Extension*

IV.2 Implementasi Kontrak Pintar

Kontrak cerdas HiFiChain.sol berhasil dikompilasi dan disebarkan (deployed) ke jaringan lokal Hardhat. Fungsi utama yang diimplementasikan mencakup pendaftaran aset, verifikasi, dan pembaruan data *oracle*.

Berikut adalah potongan kode implementasi fungsi registerAsset yang menerapkan logika kontrol akses untuk keamanan:

```
function registerAsset(string memory _model, string
memory _serial, uint256 _price) public onlyOwner {
      require(!assets[_serial].exists, "Error: Asset
already registered");


    assets[_serial] = AudioAsset({
```

```
        modelName: _model,
        serialNumber: _serial,
        manufacturedDate: block.timestamp,
        currentValueUSD: _price,
        currentOwner: msg.sender,
        exists: true
    });

            emit    AssetRegistered(_serial,    _model,
msg.sender);
}
```

Gambar IV.1 di bawah ini menunjukkan struktur kode kontrak pintar di editor teks.

```
HiFiChain.sol ×
smart-contract > contracts > HiFiChain.sol
1   // SPDX-License-Identifier: MIT
2   pragma solidity ^0.8.0;
3
4   contract HiFiChain {
5
6       struct AudioAsset {
7           string modelName;
8           string serialNumber;
9           uint256 manufacturedDate;
10          uint256 currentValueUSD;
11          address currentOwner;
12          bool exists;
13      }
14
15      mapping(string => AudioAsset) public assets;
16      uint256 public exchangeRate;
17
18      address public owner;
19
20      event AssetRegistered(string indexed serialNumber, string model, address indexed owner);
21      event RateUpdated(uint256 newRate);
22
23      modifier onlyOwner() {
24          require(msg.sender == owner, "Security Error: Caller is not the owner");
25          _;
26      }
27
28      constructor() {
29          owner = msg.sender;
```

Gambar IV.1. Struktur Kode Kontrak Pintar HiFiChain.sol

IV.3 Implementasi Oracle Service

Layanan Oracle diimplementasikan menggunakan *script* Node.js (priceOracle.js) yang berjalan secara terpisah (*off-chain*). *Script* ini secara berkala mengakuisisi data harga dari API publik CoinGecko dan memanggil fungsi `updateExchangeRate` pada kontrak pintar.

```
PS C:\Group Projects\IF4035-Tubes\oracle> node .\priceOracle.js
🤖 Oracle Service Started...
Fetching ETH Price from Internet...
✅ Current ETH Price: $1946
Pushing data to Smart Contract...
🎉 Oracle Update Complete! Data is now on-chain.
PS C:\Group Projects\IF4035-Tubes\oracle>
```

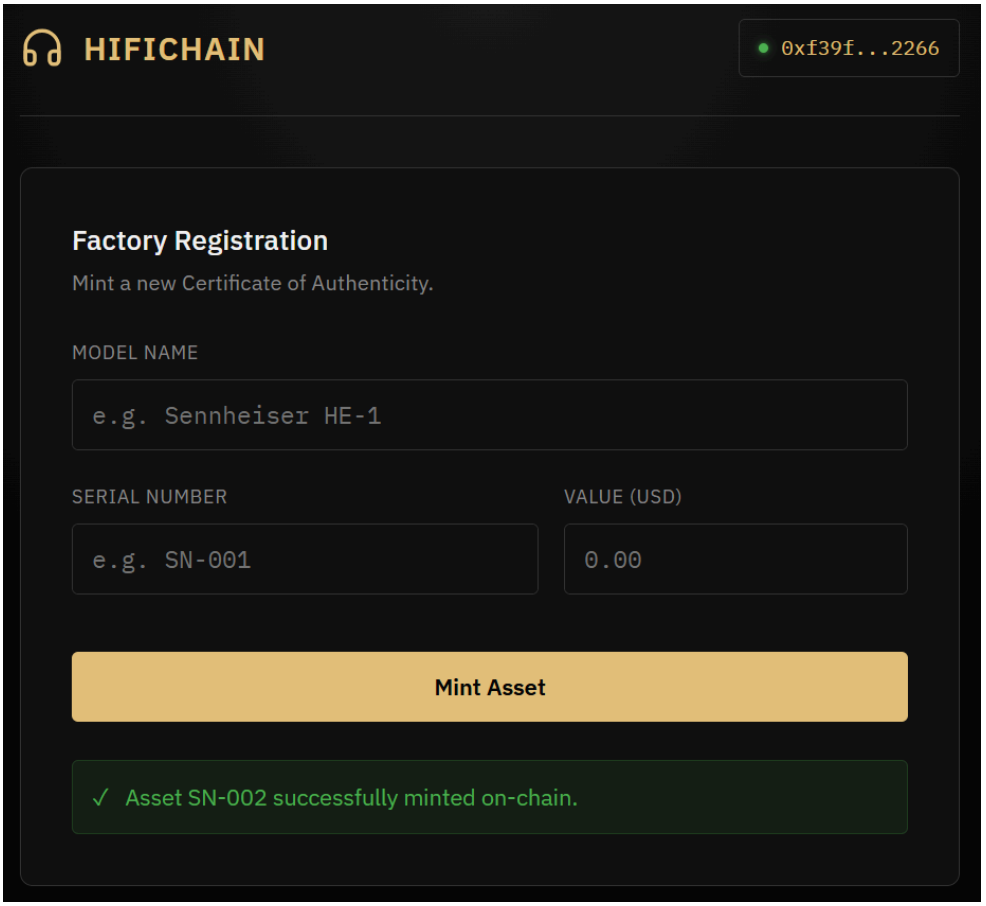
Gambar IV.2. Log Oracle Mengakuisisi Harga ETH Eksternal

IV.4 Implementasi Antarmuka Pengguna

Antarmuka pengguna dibangun menggunakan React dengan desain responsif bernuansa gelap untuk mencerminkan estetika perangkat audio premium.

IV.4.1 Halaman Pendaftaran (*Minting*)

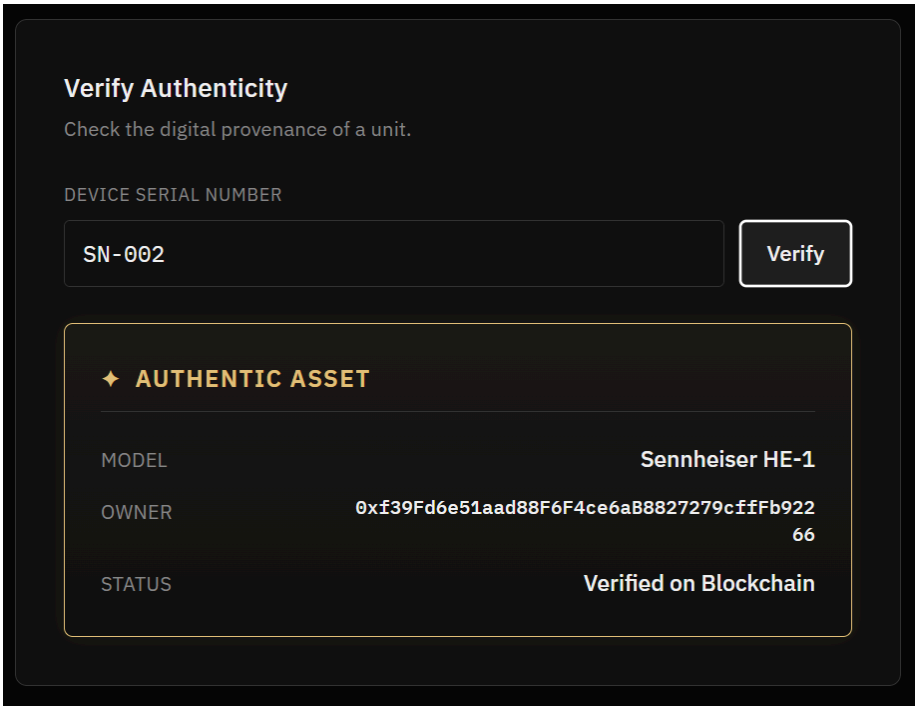
Halaman ini hanya dapat digunakan oleh pemilik kontrak (manufaktur) untuk mencetak sertifikat digital baru. Pengguna harus menghubungkan dompet MetaMask terlebih dahulu. Gambar IV.3 mengilustrasikan notifikasi sukses yang muncul pada saat pendaftaran akses berhasil dilakukan.



Gambar IV.3 Notifikasi Sukses Pendaftaran Aset

IV.4.2 Halaman Verifikasi

Halaman ini terbuka untuk umum. Pengguna memasukkan nomor seri dan sistem akan menampilkan sertifikat digital jika data ditemukan di *blockchain*. Gambar IV.4 mengilustrasikan tampilan sertifikat digital ketika aset berhasil divalidasi.



Gambar IV.4 Tampilan Sertifikat Digital Valid

BAB V

PENGUJIAN DAN ANALISIS

V.1 Skenario Pengujian (*Black Box Testing*)

Pengujian fungsional dilakukan untuk memastikan setiap fitur berjalan sesuai spesifikasi awal. Rekapitulasi hasil pengujian tercantum pada Tabel V.1.

Tabel V.1 Hasil Pengujian

No.	Fitur	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian	Status
1	Koneksi Dompet	Menekan tombol "*Connect Wallet*" pada *frontend*	MetaMask terbuka dan meminta izin koneksi	MetaMask terbuka dan akun terhubung	Berhasil
2	Registrasi Aset	Admin mendaftarkan aset dengan nomor serial baru	Transaksi berhasil, data tersimpan, notifikasi sukses muncul	Aset tersimpan *on-chain*	Berhasil
3	Cegah Duplikasi	Mendaftarkan ulang nomor serial yang sudah ada	Transaksi ditolak oleh kontrak pintar	*Revert*: "*Asset already registered*"	Berhasil
4	Keamanan Akses	Akun nonadmin mencoba mendaftarkan aset	Transaksi ditolak (*revert*) karena bukan *owner*	*Revert*: "*Caller is not the owner*"	Berhasil
5	Verifikasi Valid	Memasukkan nomor serial yang sudah terdaftar	Menampilkan detail aset (model, *owner*) dan status “Valid”	Kartu Sertifikat muncul	Berhasil

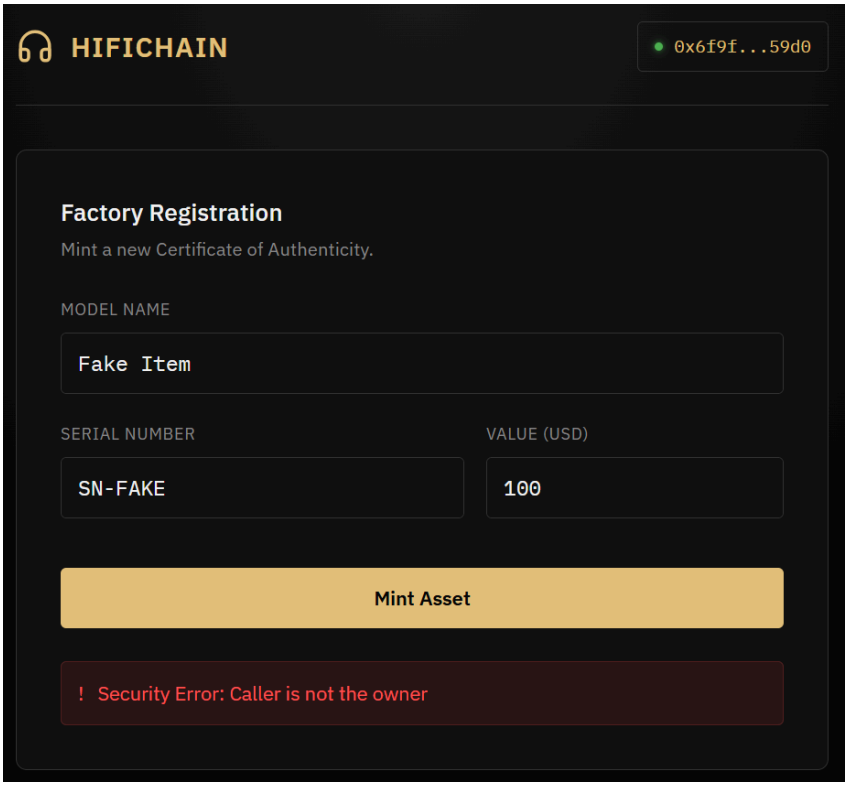
No.	Fitur	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian	Status
6	Verifikasi Invalid	Memasukkan nomor serial acak atau tidak terdaftar	Menampilkan pesan "*Verification Failed*" / "*Not Found*"	Pesan galat muncul	Berhasil

V.2 Analisis Keamanan (OWASP Smart Contract Top 10 2025)

Sistem HiFiChain telah diaudit berdasarkan standar keamanan terbaru *OWASP Smart Contract Top 10* (2025). Berikut adalah mitigasi yang diterapkan terhadap kerentanan prioritas tinggi:

V.2.1 *Access Control Vulnerabilities*

Pada versi awal pengembangan, fungsi `registerAsset` bersifat publik, yang memungkinkan siapa saja mendaftarkan aset palsu. Sebagai solusi, sistem mengimplementasikan variabel address owner dan *modifier* onlyOwner. Pengujian pada Tabel 5.1 menunjukkan hasil bahwa akun yang asing (peretas) gagal melakukan transaksi sehingga membuktikan fakta bahwa kontrol akses berfungsi.



Gambar V.1 Pesan Galat Saat Akun Tidak Berwenang

V.2.2 *Integer Overflow*

Penggunaan Solidity versi ^0.8.0 secara otomatis memberikan proteksi terhadap *integer overflow* dan *underflow* tanpa perlu pustaka tambahan (seperti SafeMath) sehingga memastikan perhitungan nilai aset dan tanggal tetap akurat.

V.2.3 Oracle Manipulation

Sistem memvalidasi input dari oracle dengan `require(_rate > 0)` untuk memastikan harga yang dimasukkan tidak bernilai nol sehingga tidak merusak logika valuasi aset.

BAB VI

PENUTUP

VI.1 Daftar Anggota dan Pembagian Tugas

Daftar anggota kelompok dan pembagian peran dalam pengerjaan tugas besar tercantum dalam Tabel VI.1.

Tabel VI.1. Daftar Anggota dan Pembagian Tugas

NIM	Nama	Pembagian Tugas
13521154	Naufal Baldemar Ardanni	● Pengembangan kontrak pintar ● Integrasi *oracle* ● Pengembangan *frontend* ● Penulisan Laporan, Manual, dan *Paper* ● Pembuatan Video Demo dan Presentasi

VI.2 Saran Pengembangan

Selama pengerjaan proyek ini, beberapa kendala teknis dan pembelajaran yang didapat meliputi:

1. **Kendala Sinkronisasi *Nonce***: Saat mereset jaringan Hardhat lokal, MetaMask sering mengalami ketidakcocokan *nonce* transaksi. Solusi atas kendala ini adalah melakukan reset akun secara manual pada pengaturan MetaMask tiap kali jaringan di-*restart*.
2. **Kompleksitas Asinkron**: Menangani interaksi *frontend* dengan *blockchain* yang bersifat asinkron (menunggu blok di-*mining*) memerlukan pengelolaan *state* React yang hati-hati untuk memberikan umpan balik antarmuka (UI) yang responsif (*Loading*/*Success*/*Error*).
3. **Pembelajaran Utama**: Pemahaman mendalam mengenai batasan isolasi *blockchain* (mengapa Oracle diperlukan) dan pentingnya validasi input pada sisi *on-chain* meskipun *frontend* sudah divalidasi.

LAMPIRAN

Repositori kode sumber perangkat lunak dapat diakses pada tautan berikut: https://github.com/NaufalArdanni/HiFiChain.

Daftar putar (*playlist*) video demo dan presentasi dapat diakses pada tautan berikut: https://www.youtube.com/playlist?list=PLOP7MIysOlx9QSCsXAPLbJmS8Abl42myj.

DAFTAR PUSTAKA

Al-Breiki, H., Rehman, M. H. U., Salah, K., dan Svetinovic, D. (2020): Trustworthy Blockchain Oracles: Review, Comparison, and Open Research Challenges, IEEE Access, 8, 85675–85685. https://doi.org/10.1109/access.2020.2992698

Atzei, N., Bartoletti, M., dan Cimoli, T. (2017): A Survey of Attacks on Ethereum Smart Contracts (SoK), Lecture Notes in Computer Science, Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-662-54455-6_8

Buterin, V. (2014). A Next-Generation Smart Contract and Decentralized Application Platform. Ethereum White Paper, dari https://ethereum.org/en/whitepaper/

Hardhat. Hardhat Documentation: Ethereum Development Environment. Nomic Foundation, dari https://hardhat.org/docs

OWASP Foundation. (2025). OWASP Smart Contract Top 10. OWASP, dari https://owasp.org/www-project-smart-contract-top-10/

Solidity Team. Solidity v0.8.28: Documentation. Ethereum Foundation, dari https://docs.soliditylang.org/

Toyoda, K., Mathiopoulos, P. T., Sasase, I., dan Ohtsuki, T. (2017): A Novel Blockchain-Based Product Ownership Management System (POMS) for Anti-Counterfeits in the Post Supply Chain, IEEE Access, 5, 17465–17477. https://doi.org/10.1109/access.2017.2720760

Wood, G. (2014). Ethereum: A secure decentralised generalised transaction ledger. Ethereum project yellow paper, 151(2014), 1-32.